DNAlity

Documentation Technique

Application web de statistiques musicales en temps réel

Lisa ANTON • Paulo GRIENENBERGER

Ynov Campus Bordeaux — B1 Informatique — 2025/2026

Sommaire

1. Présentation du projet
2. Architecture technique
3. Plan d'adressage réseau
4. Choix techniques
5. Infrastructure et virtualisation
6. Mise en place de l'environnement
7. Sécurisation de l'environnement
8. Frontend — Interface utilisateur
9. Backend — API FastAPI
10. Base de données Supabase
11. Procédures de déploiement
12. Pistes d'amélioration

1. Présentation du projet

1.1 Contexte

DNAlity est une application web développée dans le cadre d'un projet infra de fin d'année en B1 Informatique à Ynov Campus Bordeaux. Elle permet à tout utilisateur de consulter ses statistiques musicales personnelles en temps réel, directement depuis son compte Spotify.

Le projet répond à un besoin simple : offrir une expérience similaire au "Spotify Wrapped" annuel, mais accessible à tout moment, avec un sélecteur de période (aujourd'hui, 4 semaines, 6 mois, depuis toujours).

1.2 Équipe

Membre	Rôle	Responsabilités principales
Paulo Grienenberger	Backend Developer	API FastAPI, OAuth Spotify, Supabase, routes de données
Lisa Anton	Frontend Developer	Interface HTML/CSS, dashboard, page d’erreur 404, intégration API

1.3 Fonctionnalités

- Connexion OAuth 2.0 via le compte Spotify de l'utilisateur
- Dashboard personnalisé avec top artistes, top titres, genres favoris
- Statistiques par période : aujourd'hui (après 21h), 4 semaines, 6 mois, depuis toujours
- Affichage des titres récemment écoutés avec heure exacte
- Calcul automatique des minutes écoutées et du nombre d'artistes uniques (pour la journée seulement)
- Page d'erreur 404 personnalisée
- Animations visuelles : égaliseur, notes de musique, compteurs animés

1.4 Périmètre du projet

L’intégration Deezer était initialement prévue dans le plan du projet. Cependant au moment du développement son API n’était pas disponible, alors impossible de récupérer les données des utilisateurs. L’intégration de Deezer a donc été reportée dans les pistes d’amélioration futures.

2. Architecture technique

2.1 Vue d'ensemble

L'application repose sur une architecture client-serveur répartie sur deux machines virtuelles VirtualBox sous Windows. Le frontend et le backend sont développés et hébergés séparément, et communiquent via des appels HTTP REST.

Composant	Technologie	Machine	Port
Frontend	HTML / CSS / JavaScript	VM2 — Lisa (10.33.77.90)	5500 (Live Server)
Backend	Python / FastAPI / Uvicorn	VM1 — Paulo (10.33.69.115)	8000
Base de données	Supabase (PostgreSQL)	Cloud Supabase	443 (HTTPS)
API externe	Spotify Web API	Serveurs Spotify	443 (HTTPS)

2.2 Schéma de communication

Le flux de données suit ce chemin lors d'une connexion utilisateur :

- L'utilisateur ouvre le frontend sur VM2 et clique sur "Continuer avec Spotify"
- Le frontend appelle GET /spotify/auth sur le backend (VM1:8000)
- Le backend génère l'URL d'autorisation Spotify et la retourne
- L'utilisateur est redirigé vers la page de login Spotify
- Spotify redirige vers GET /spotify/callback avec un code d'autorisation
- Le backend échange le code contre un access token et le sauvegarde dans Supabase
- Le frontend charge le dashboard et appelle les routes de données
- Le backend interroge l'API Spotify avec le token et retourne les statistiques

2.3 Structure des fichiers

Dossier / Fichier	Description
frontend/index.html	Page d'accueil et connexion Spotify
frontend/style.css	Esthétique de la page d’acceuil
frontend/dashboard.html	Dashboard des statistiques
frontend/dashboard.css	Esthétique du dashboard
frontend/erreur404.html	Page d'erreur personnalisée

frontend/erreur404.css	Esthétique de la page d'erreur
backend/main.py	Point d'entrée FastAPI, configuration CORS
backend/services/spotify.py	Routes OAuth et données Spotify
backend/requirement.txt	Dépendances Python pour lancer le serveur
.env	Variables d'environnement (non versionné)
.gitignore	Fichiers exclus du dépôt Git

3. Plan d'adressage réseau

3.1 Topologie

Le projet est déployé sur un réseau local. Chaque développeur travaille sur sa propre machine Windows physique, avec sa propre branche (une pour le back et une pour le front) qui héberge une machine virtuelle VirtualBox dédiée à son composant applicatif. Les deux machines communiquent sur le réseau local de l'établissement.

3.2 Configuration réseau VirtualBox

Les machines virtuelles utilisent le mode réseau NAT de VirtualBox, permettant l'accès à internet tout en partageant l'adresse IP de la machine hôte. La communication entre frontend et backend s'effectue via les adresses IP physiques des machines hôtes.

4. Choix techniques

4.1 Tableau des choix

Composant	Technologie choisie	Justification
Frontend	HTML / CSS	Simplicité, pas de dépendances, maîtrise en B1
Backend	Python / FastAPI	Syntaxe claire, documentation automatique via /docs, async natif
Base de données	Supabase (PostgreSQL)	Hébergement cloud gratuit, client Python disponible, temps réel
Virtualisation	VirtualBox	Logiciel libre, disponible sur Windows, familier

Versionnement	Git / GitHub	Standard industrie, collaboration simplifiée
Auth streaming	Spotify OAuth 2.0	API bien documentée, scopes précis, SDK disponible
Serveur web dev	Uvicorn + Live Server	Adapté au développement local, rechargement automatique

4.2 Dépendances Python

Package	Version	Rôle
fastapi	0.115.0	Framework web API REST
uvicorn	latest	Serveur ASGI pour FastAPI
pydantic	2.11.0	Validation des données et modèles
supabase	2.10.0	Client Python pour Supabase
httpx	0.27.0	Requêtes HTTP asynchrones
python-dotenv	latest	Chargement des variables d'environnement
requests	latest	Requêtes HTTP vers l'API Spotify

5. Infrastructure et virtualisation

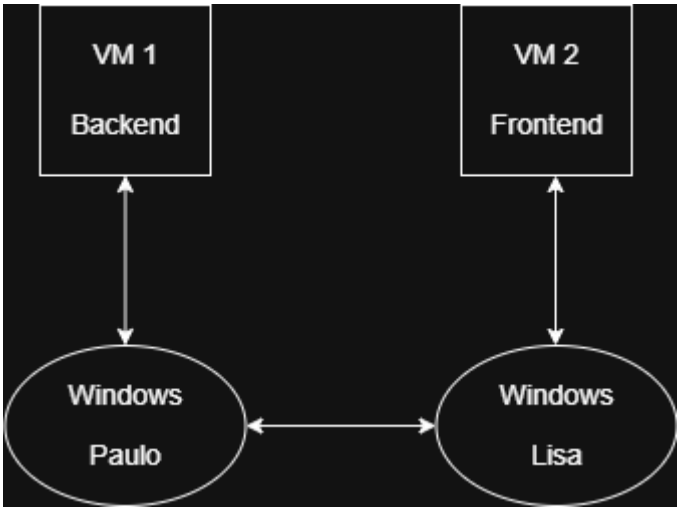
5.1 Configuration des machines virtuelles

Chaque développeur a mis en place une machine virtuelle VirtualBox sur son poste Windows personnel pour isoler l'environnement de développement du système hôte.

Paramètre	VM1 — Backend (Paulo)	VM2 — Frontend (Lisa)
Système hôte	Windows	Windows
Logiciel de virtualisation	VirtualBox	VirtualBox
Mode réseau	NAT	NAT
Port applicatif	8000 (Uvicorn)	5500 (Live Server)
Langage	Python 3.12	HTML / CSS
Éditeur	VS Code	VS Code

5.2 Schéma réseau

Le schéma ci-dessous illustre la topologie du projet avec les deux machines Windows physiques qui communique et leurs VM respectives.



6. Mise en place de l'environnement

6.1 Prérequis

- Python 3.12 installé (avec "Add Python to PATH" coché)
- VS Code avec l'extension Live Server
- VirtualBox installé
- Compte GitHub pour le versionnement
- Compte Supabase avec projet créé
- Application déclarée sur le Spotify Developer Dashboard

6.2 Installation du backend

Cloner le dépôt et installer les dépendances :

Étape	Commande
1. Cloner le projet	git clone https://github.com/[repo]/dnality.git
2. Aller dans le dossier	cd dnality
3. Installer les dépendances	pip install -r backend/requirement.txt
4. Créer le fichier .env	Voir section 6.3
5. Lancer le backend	uvicorn backend.main:app --reload

6.3 Configuration du fichier .env

Créer un fichier .env à la racine du projet avec les variables suivantes :

Variable	Description	Où la trouver
SPOTIFY_CLIENT_ID	Identifiant de l'application Spotify	Spotify Developer Dashboard
SPOTIFY_CLIENT_SECRET	Clé secrète Spotify	Spotify Developer Dashboard
SUPABASE_URL	URL du projet Supabase	Paramètres du projet Supabase
SUPABASE_KEY	Clé API Supabase (anon key)	Paramètres du projet Supabase

6.4 Lancement du frontend

Ouvrir VS Code, faire un clic droit sur frontend/index.html, puis choisir "Open with Live Server". Le site sera accessible sur http://127.0.0.1:5500.

6.5 Vérification du bon fonctionnement

Test	URL	Résultat attendu
Santé du backend	http://127.0.0.1:8000/health	{"status": "ok"}
Documentation API	http://127.0.0.1:8000/docs	Interface Swagger FastAPI
Route auth Spotify	http://127.0.0.1:8000/spotify/auth	JSON avec auth_url
Frontend	http://127.0.0.1:5500	Page d'accueil DNAlity

7. Sécurisation de l'environnement

7.1 Gestion des secrets

Les informations sensibles (clés API, secrets Spotify, credentials Supabase) ne sont jamais écrites en dur dans le code source. Elles sont stockées dans un fichier .env qui est explicitement exclu du dépôt Git via le fichier .gitignore.

- Fichier .env listé dans .gitignore
- Variables chargées au runtime via python-dotenv
- Aucune clé ne figure dans l'historique Git

7.2 OAuth 2.0 Spotify

L'authentification utilise le protocole OAuth 2.0 d'autorisation par code (Authorization Code Flow). L'utilisateur ne transmet jamais son mot de passe Spotify à l'application — il se connecte directement sur les serveurs de Spotify.

- Le frontend ne manipule aucun secret
- L'access token est stocké côté backend dans Supabase
- Un mécanisme de refresh token permet de renouveler l'accès sans reconnexion

7.3 Configuration CORS

Le middleware CORS est configuré dans main.py pour contrôler les origines autorisées à communiquer avec le backend. En développement, toutes les origines sont acceptées pour faciliter les tests.

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

7.4 Scopes Spotify demandés

Scope	Permission accordée	Utilisation dans l'app
user-read-email	Lecture de l'email du compte	Identification de l'utilisateur
user-read-private	Lecture du profil (nom, pays, abonnement)	Affichage du prénom sur le dashboard
user-top-read	Lecture des top artistes et titres	Sections Top Artistes et Top Titres
user-read-recently-played	Lecture des titres récemment joués	Stats du jour, titres récents, KPI

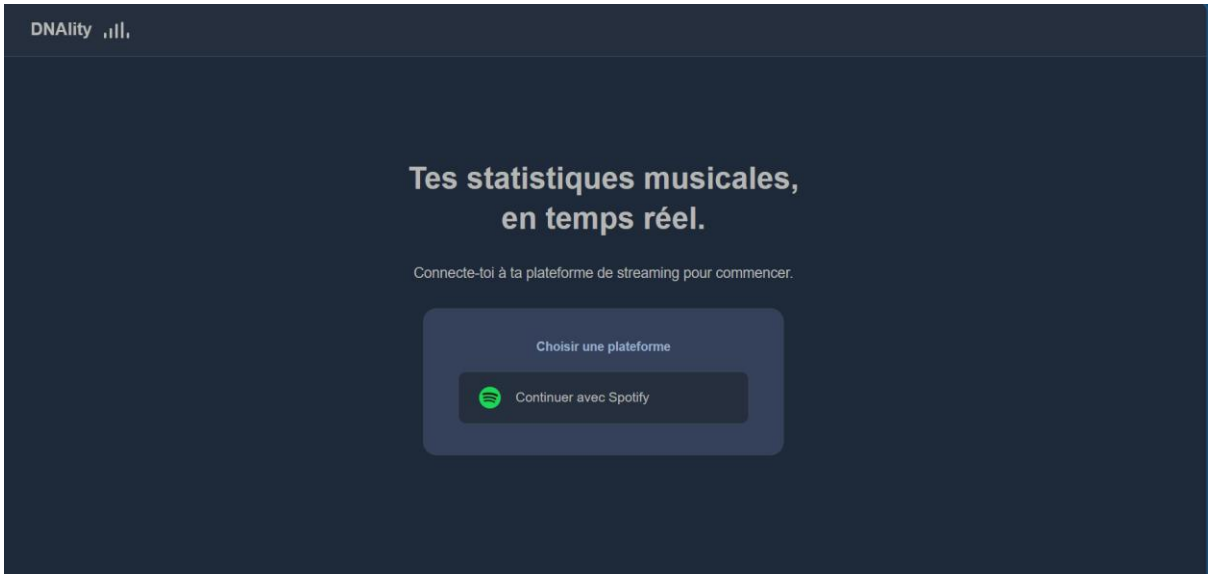
8. Frontend — Interface utilisateur

8.1 Pages de l'application

Fichier	URL	Description
index.html	/	Page d'accueil avec bouton de connexion Spotify
dashboard.html	/dashboard.html	Dashboard principal des statistiques
erreur404.html	/erreur404.html	Page d'erreur personnalisée

8.2 Page d'accueil

La page d'accueil présente DNAlity avec un titre, un sous-titre, et un bouton "Continuer avec Spotify". Au clic, le JavaScript appelle GET /spotify/auth sur le backend pour récupérer l'URL d'autorisation Spotify, puis redirige l'utilisateur.



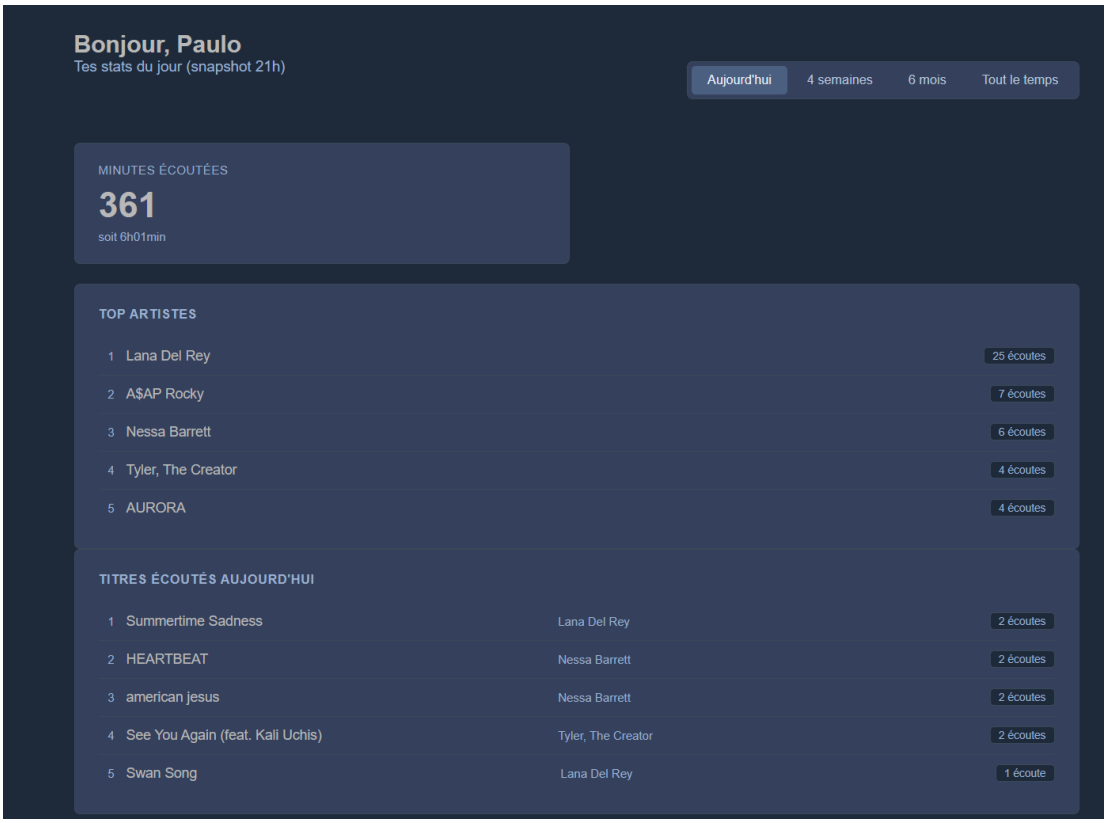
8.3 Dashboard

Le dashboard est la page principale de l'application. Il est composé de :

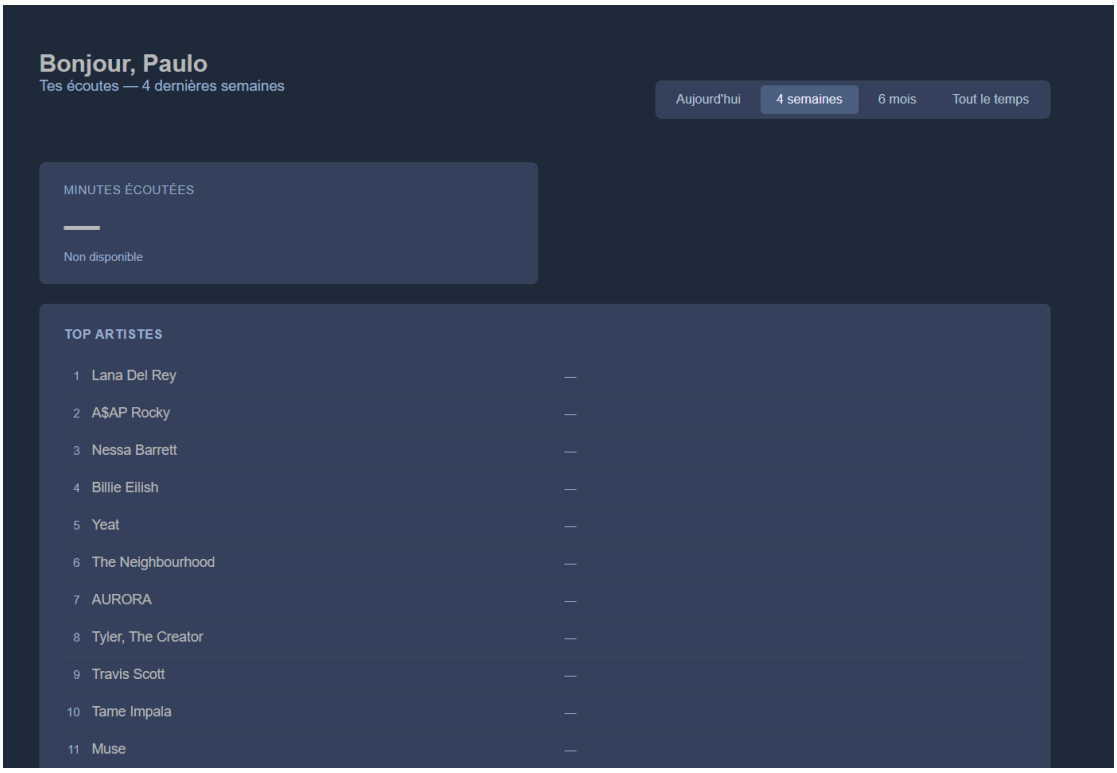
- Un header avec le logo, un égaliseur animé CSS, le nom d'utilisateur et un bouton de déconnexion
- Un sélecteur de période : Aujourd'hui, 4 semaines, 6 mois, Tout le temps
- 4 cartes KPI : minutes écoutées, titres joués, artistes différents, dernier titre joué
- Section Top Artistes avec barres de popularité animées
- Section Top Titres avec album et durée
- Section Genres Favoris calculés depuis les artistes
- Section Titres récents avec heure exacte (vue Aujourd'hui uniquement)

- Un fond animé avec des notes de musique qui montent

Statistiques du jour



Statistique sur 4 semaines



8.4 Page erreur 404

Nous avons personnalisé la page d'erreur 404 pour qu'elle soit en accord avec le thème du site. Lorsqu'un problème survient, par exemple lorsque l'API ne répond pas ou que le dashboard n'est pas accessible, l'utilisateur est redirigé vers cette page et peut revenir à l'accueil du site.

8.5 Règle de 21h

Après 21h, les statistiques de la journée sont affichées. Avant 21h, ce sont les statistiques de la veille qui sont affichées. Ce choix vise à présenter des données complètes et représentatives d'une journée d'écoute.

8.6 Appels API depuis le frontend

Route appelée	Méthode	Données retournées
/spotify/auth	GET	URL de connexion Spotify
/spotify/me	GET	Nom d'utilisateur
/spotify/top-artists	GET	Top 50 artistes
/spotify/top-tracks	GET	Top 50 titres
/spotify/recently-played	GET	Top 5 derniers titres avec horodatage

9. Backend — API FastAPI

9.1 Structure de l'API

Le backend est une API REST développée avec FastAPI. Il est lancé avec Uvicorn sur le port 8000 et expose plusieurs endpoints organisés sous le préfixe /spotify.

9.2 Routes disponibles

Route	Méthode	Description
/health	GET	Vérification que le serveur est actif
/spotify/auth	GET	Génère l'URL d'autorisation OAuth Spotify
/spotify/callback	GET	Reçoit le code Spotify, échange contre un token, sauvegarde dans Supabase
/spotify/refresh	GET	Rafraîchit le token d'accès expiré
/spotify/me	GET	Retourne le profil de l'utilisateur connecté
/spotify/top-artists	GET	Retourne les top artistes (short_term par défaut)
/spotify/top-tracks	GET	Retourne les top titres (short_term par défaut)
/spotify/recently-played	GET	Retourne les 50 derniers titres écoutés

9.3 Configuration du serveur

Le fichier main.py configure l'application FastAPI avec le middleware CORS et inclut les routers. La commande de lancement depuis la racine du projet est :

```
uvicorn backend.main:app --reload
```

9.4 Gestion des tokens

Après la connexion OAuth, le token Spotify est sauvegardé dans la table spotify_token de Supabase avec les champs access_token, refresh_token et expires_at. Avant chaque appel à l'API Spotify, le token peut être rafraîchi via la route /spotify/refresh si nécessaire.

10. Base de données Supabase

10.1 Présentation

Supabase est une alternative open-source à Firebase basée sur PostgreSQL. Le projet utilise l'offre cloud gratuite de Supabase pour stocker les tokens d'authentification Spotify.

10.2 Structure de la base

Une seule table est utilisée pour cette version du projet :

Colonne	Type	Description
id	text (PK)	Identifiant de l'utilisateur ("user_1" pour cette version)
access_token	text	Token d'accès Spotify (valide 1 heure)
refresh_token	text	Token de rafraîchissement (longue durée)
expires_at	integer	Timestamp Unix d'expiration du token

10.3 Connexion depuis Python

Le client Supabase Python est initialisé dans spotify.py à partir des variables d'environnement SUPABASE_URL et SUPABASE_KEY. Les opérations utilisées sont upsert (insertion ou mise à jour) pour sauvegarder les tokens et select avec filtre pour les lire.

12. Procédures de déploiement

12.1 Procédure de démarrage complète

N°	Responsable	Action
1	Paulo	Ouvrir VS Code sur la VM1
2	Paulo	Ouvrir un terminal à la racine du projet
3	Paulo	Vérifier que le fichier .env est présent

4	Paulo	Exécuter : uvicorn backend.main:app --reload
5	Paulo	Vérifier : http://127.0.0.1:8000/health renvoie {"status": "ok"}
6	Lisa	Ouvrir VS Code sur la VM2
7	Lisa	Clic droit sur frontend/index.html → Open with Live Server
8	Lisa	Ouvrir http://127.0.0.1:5500 dans le navigateur
9	Lisa	Cliquer sur "Continuer avec Spotify" et vérifier la redirection
10	Lisa	Se connecter avec un compte Spotify et vérifier le dashboard

12.2 Procédure de mise à jour du code

- Récupérer les dernières modifications : git pull origin main
- Si de nouveaux packages ont été ajoutés : pip install -r backend/requirement.txt
- Relancer le serveur : uvicorn backend.main:app --reload

13. Pistes d'amélioration

13.1 Fonctionnalités

- Intégration de la plateforme Deezer via l'API officielle
- Support multi-utilisateurs avec gestion de sessions individuelles
- Comparaison des statistiques entre deux périodes

13.3 UX / Design

- Version mobile responsive
- Mode clair / mode sombre

Conclusion

DNAlity est une application web fonctionnelle qui permet à un utilisateur Spotify de consulter ses statistiques musicales personnelles en temps réel depuis un dashboard interactif.

Le projet a permis de mettre en pratique les notions de réseau (architecture client-serveur, adressage IP, communication inter-VM), de développement web (frontend HTML/CSS), backend Python/FastAPI), de base de données (Supabase), de sécurité (OAuth 2.0, gestion des secrets) et de virtualisation (VirtualBox).